Auto

Upgrade

Nov 12

math_rush

my code when ran in thonny does not display the game:









I can see the file. The most likely issue is that your program depends on the Drafter web framework:

```
from drafter import *
```

and it starts with:

```
start_server(initial_state)
```

This does not create a normal graphics window in Thonny. Instead, Drafter starts a local web server and displays the game in a web browser.

A few things to check:

1. Look for errors in the Shell

When you run the program, does Thonny display an error such as:

```
ModuleNotFoundError: No module named 'drafter'
```

or

```
ModuleNotFoundError: No module named 'bakery'
```

If so, those libraries are not installed.

2. Check whether a browser tab opens

When `start_server(initial_state)` runs successfully, you should see a message similar to:

```
Running on http://127.0.0.1:...
```

Then a browser should open automatically. If it doesn't, copy the URL from the Shell and paste it into your browser.

3. Verify Drafter is installed

In Thonny's Shell, try:

```
import drafter
```

```
print("drafter works")
```

If that fails, you need to install Drafter.

4. Verify Bakery is installed

Your code also imports:

```
from bakery import assert_equal
```

Try:

```
import bakery

print("bakery works")
```

5. Possible Unicode issue

Your questions use symbols like:

```
"×"

"÷"
```

These are usually fine, but if you get encoding-related errors, replace them temporarily with:

```
"*"
```

```
"/"
```

6. Test whether the server is reached

Replace the last line with:

```
print("Starting server...")
```

```
start_server(initial_state)
```

If you see:

```
Starting server...
```

and then nothing else, the program may be waiting for the browser connection.

Can you paste the exact output/error message from Thonny's Shell when you run the file? That will pinpoint the problem immediately.

my code when ran in thonny does not display the game when opened in drafter:

Looking through your code, I don't see anything that would automatically prevent the Drafter app from starting. The biggest clue is that you say it runs in Thonny but nothing displays in Drafter.

A few likely causes:

1. The server is starting, but you're on the wrong URL

Your last line is:

```
start_server(initial_state)
```

When you run the file, Drafter should print something like:

```
Now visiting http://127.0.0.1:5000
```

or a similar localhost address.

Make sure you're opening the exact URL printed in the shell, not an old Drafter tab.

---

2. One of your route pages is crashing

A potential issue is:

```
Header(current_question.prompt, 2)
```

Depending on your Drafter version, `Header` may only accept one argument. Try changing it to:

```
Header(current_question.prompt)
```

and similarly:

```
Header("Previous Games", 2)
```

to

```
Header("Previous Games")
```

---

3. Check for Shell errors

Your code imports:

```
from drafter import *
```

```
from bakery import assert_equal
```

If either module is missing, the server never starts.

Look for errors such as:

```
ModuleNotFoundError
```

```
TypeError
```

```
AttributeError
```

in the Thonny Shell.

4. Verify the home page loads

Temporarily replace everything after the dataclasses with:

```
from drafter import *




@route

def home_page(state):

   return Page(state, [

       "Hello World"

   ])




start_server({})
```

If this appears in the browser, then Drafter itself is working and the problem is somewhere in the game code.

---

5. Possible issue with `SelectBox`

Some Drafter versions expect:

```
SelectBox("difficulty", options=DIFFICULTIES)
```

rather than:

```
SelectBox("difficulty", DIFFICULTIES, state.difficulty)
```

If the Settings page is the first page that fails, this is worth checking.

---

6. Most suspicious issue in your code

Your routes are defined as:

```
@route


def home_page(state: State) -> Page:
```

but there is no route explicitly specified as the starting page.

Many Drafter projects end with:

```
start_server(initial_state, home_page)
```

instead of:

```
start_server(initial_state)
```

If your version of Drafter requires the starting route, the browser may open to a blank page.

Try:

```
start_server(initial_state, home_page)
```

The fastest way to find the exact problem is to copy the entire error message shown in the Thonny Shell after running the program. That will usually identify the specific line that's preventing the page from rendering.

```
SUCCESS at line 166 (assert_equal)
SUCCESS at line 167 (assert_equal)
SUCCESS at line 168 (assert_equal)
SUCCESS at line 169 (assert_equal)
SUCCESS at line 433 (assert_equal)
SUCCESS at line 434 (assert_equal)
SUCCESS at line 435 (assert_equal)
SUCCESS at line 454 (assert_equal)
SUCCESS at line 455 (assert_equal)
SUCCESS at line 474 (assert_equal)
SUCCESS at line 475 (assert_equal)
Drafter noticed that math_rush.py is in a large or broad folder.
For safety, live reload will watch your program and files used by
your site rather than the entire folder.
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Drafter ready at http://localhost:8000
INFO:     ::1:56914 - "GET / HTTP/1.1" 200 OK
INFO:     ::1:63400 - "WebSocket /__drafter_ws" [accepted]
INFO:     connection open
INFO:     ::1:56914 - "GET /math_rush.py HTTP/1.1" 304 Not Modified
```

The good news is that your output shows Drafter is starting successfully:

```
INFO: Application startup complete.
```

```
INFO: Drafter ready at http://localhost:8000


INFO: GET / HTTP/1.1" 200 OK
```

That means:

✅ Your tests passed

✅ No Python syntax errors

✅ No crashes in `start_server(initial_state)`

✅ Drafter is serving a page

So the problem is likely the page is rendering blank or failing on the frontend.

First thing I'd check

Your route contains:

```
Header(current_question.prompt, 2)
```

and later:

```
Header("Previous Games", 2)
```

Many Drafter versions don't accept a second argument for `Header`.

Try changing them to:

```
Header(current_question.prompt)
```

and

```
Header("Previous Games")
```

Then rerun.

Another likely issue

Your server starts with:

```
start_server(initial_state)
```

Some versions of Drafter require:

```
start_server(initial_state, home_page)
```

Try changing the last line to:

```
start_server(initial_state, home_page)
```

Open the browser developer console

Since the Python server isn't showing any errors, the error may be occurring in the browser.

1. Open `http://localhost:8000`
2. Press F12
3. Click Console
4. Refresh the page

If you see a red error, paste it here.

Quick test

Replace `home_page` temporarily with:

```
@route

def home_page(state: State) -> Page:

   return Page(state, [

       "Hello World"

   ])
```

If "Hello World" appears, then the issue is one of the components on your real page (Header, Button, SelectBox, etc.).

One thing that stands out

You're using:

```
SelectBox("difficulty", DIFFICULTIES, state.difficulty)
```

```
SelectBox("rounds", ROUND_OPTIONS, str(state.total_rounds))


SelectBox("timer", TIMER_OPTIONS, str(state.timer_limit))
```

Different Drafter versions have different `SelectBox` signatures. If the page goes blank specifically when opening the Settings page, these are prime suspects.

Since the server is clearly running, I suspect the problem is either:

1. `Header(..., 2)` causing a frontend render error, or
2. `SelectBox(...)` arguments not matching your Drafter version.

Can you tell me what you see at `http://localhost:8000`?

- completely blank white page
- page loads but no buttons
- "Internal Server Error"
- spinning/loading forever

That will narrow it down quickly.

Message Copilot

Copilot is an AI and may make mistakes.

change the code to fix the header issue

Yes. Replace the two `Header` calls that pass a second argument.

Change #1

From:

```
Header(current_question.prompt, 2),
```

to:

```
Header(current_question.prompt),
```

in `game_page`.

Change #2

From:

```
Header("Previous Games", 2),
```

to:

```
Header("Previous Games"),
```

in `leaderboard_page`.

The modified sections should look like this:

```
@route
def game_page(state: State) -> Page:
    if len(state.questions) == 0:
        return settings_page(state)


    current_question = state.questions[-1]


    return Page(state, [
        Header("MATH RUSH - Round " + str(state.current_round)),
        "Difficulty: " + state.difficulty,
```

```
        "Question " + str(state.current_round) + " of "

        + str(state.total_rounds),

        Header(current_question.prompt),

        TextBox("answer", ""),

        Button("Submit Answer", "submit_answer"),

        Button("I Ran Out of Time", "timeout"),

        "Score: " + str(state.score),

        "Current streak: " + str(state.streak),

        "Time allowance: " + str(state.timer_limit) + " seconds",

        Button("Quit to Home", "home_page"),

    ])
```

and:

```
@route

def leaderboard_page(state: State) -> Page:

    content = [

        Header("Leaderboard"),
```

```
        "Best score: " + str(state.best_score),

        "Longest streak: " + str(state.best_streak),

        "Games played: " + str(state.games_played),

        Header("Previous Games"),

    ]
```

If it still loads a blank page after this change, the next thing I'd investigate is the `SelectBox(...)` calls, since Drafter versions sometimes differ on their expected arguments.

all drafter says is

```
Hello world!
Welcome to Drafter.
```

That message is the key clue.

If Drafter shows:

```
Hello world!

Welcome to Drafter.
```

then your `math_rush.py` isn't the file Drafter is serving as the homepage.

Your server log shows:

```
GET / HTTP/1.1" 200 OK
```

```
GET /math_rush.py HTTP/1.1" 304 Not Modified
```

which suggests Drafter is running, but the root page (`/`) is still using Drafter's default welcome page instead of your routes.

Try these checks:

1. Make sure you're running the correct file

In Thonny, verify that the tab you're running is:

```
math_rush.py
```

and not another file.

---

2. Check the final line

It should be:

```
start_server(initial_state)
```

or

```
start_server(initial_state, home_page)
```

and it should actually execute.

Add:

```
print("STARTING MATH RUSH")

start_server(initial_state)
```

If you don't see:

```
STARTING MATH RUSH
```

in the shell, then the file being run is not the one you think it is.

---

3. Force the starting page

Try replacing:

```
start_server(initial_state)
```

with

```
start_server(initial_state, home_page)
```

4. Verify the route registration

Temporarily change your home page to:

Python
@route
def home_page(state: State) -> Page:
return Page(state, [
Header("MATH RUSH TEST"),
"If you see this, the routes are working."
])

Then restart the server completely.

If you still see:

Plain Text
Hello world!
Welcome to Drafter.

then Drafter is definitely not loading your `home_page` route at all.

---

5. Fully stop and restart

Sometimes Drafter keeps an old server running.

1. Stop the program in Thonny.
2. Close the browser tab.
3. Run `math_rush.py` again.
4. Open the exact URL shown in the shell.

The strongest evidence right now is that **Drafter is serving its default landing page instead of your `home_page` route**, which usually means either:

- the wrong file is being run,
- `start_server` needs a starting route (`home_page`),
- or an old Drafter server instance is still running.

Try:

Python
start_server(initial_state, home_page)

first. That's the most likely fix.

4. Verify the route registration Temporarily change your home page to: Python @route def home_page(state: State) -> Page: return Page(state, [Header("MATH RUSH TEST"), "If you see this, the routes are working."]) Then restart the server completely. If you still see: Plain Text Hello world! Welcome to Drafter. then Drafter is definitely not loading your route at all.

i still saw the same message

That's actually a very important result.

If you changed `home_page` to:

Python
@route
def home_page(state: State) -> Page:
return Page(state, [
Header("MATH RUSH TEST"),
"If you see this, the routes are working."
])

and Drafter **still** displays:

Plain Text
Hello world!
Welcome to Drafter.

then the problem is **not in your routes**.

The problem is that **Drafter is not running the code in `math_rush.py` at all**, even though the server starts.

Let's verify what's happening.

Add this immediately above:

Python
start_server(initial_state)

Python
print("RUNNING MATH_RUSH.PY")

Then run the file.

Do you see:

Plain Text
RUNNING MATH_RUSH.PY

in the shell?

If YES

Then the file is running, but Drafter is ignoring your routes. In that case, change:

Python
start_server(initial_state)

to

Python
start_server(initial_state, home_page)

and rerun.

If NO

Then Thonny is running a different file than the one you're editing.

Another possibility is that you're opening the wrong page.

Your log contains:

Plain Text
GET / HTTP/1.1 200 OK
GET /math_rush.py HTTP/1.1 304 Not Modified

Try visiting:

```
None
http://localhost:8000/math_rush.py
```

instead of:

```
None
http://localhost:8000/
```

Some Drafter setups serve the app from the filename route rather than the root route.

One more test:

Replace **everything** in the file with only:

Python
from drafter import *

@route
def home_page(state):
return Page(state, [
"THIS IS MY TEST APP"
])

start_server({})

Run it.

If you still get:

Plain Text
Hello world!
Welcome to Drafter.

then you're definitely looking at a different Drafter app/server than the one created by `math_rush.py`.

If that's the case, tell me:

1. The exact command you use to run the program.
2. Whether `print("RUNNING MATH_RUSH.PY")` appears in the shell.
3. What appears when you visit `http://localhost:8000/math_rush.py`.

Those three answers will pinpoint the issue.

double check that my code meets all of the following requirements, flag which ones fail if any:

[redacted]

Site Functionality Requirements
Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

[redacted]

- At least 7 routes.
- At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
- The implementation of the *State* dataclass must have at least 4 fields.
  - At least one field must be a list of class instances or a dictionary of class instances.
    - This means that there must be a second dataclass.
  - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (global constantsLinks to an external site. are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements
Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in the syllabus.
Once you are finished with your project code, check over the following:

[redacted]

Chosen good variable names (check last two sections of Chapter 1B1Links to an external site.)
Unit tested your functions where possible, being valid and thoroughLinks to an external site.: each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
- If a function can't be unit tested, you must note as much in a docstring or comment.

Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
Included types in your function headers
Included good docstrings Links to an external site.as the first line of your function's body

[redacted]

here is my code:

[redacted]

[redacted]

[redacted]

[redacted]

- Decomposed complicated functions with helper functions

Based on the code you provided, here's a requirement-by-requirement audit.

Site Functionality Requirements

✅ 1. At least 7 routes

You have 9 routes:

```
home_page

instructions_page

settings_page

start_game

game_page

submit_answer

timeout

result_page

next_round

summary_page

leaderboard_page
```

Pass.

✅ 2. At least 5 Pages

You have far more than 5 pages:

- Home
- Instructions
- Settings
- Game
- Results
- Summary
- Leaderboard

Pass.

✅ 3. State dataclass has at least 4 fields

Your `State` dataclass has 13 fields.

Pass.

✅ 4a. At least one field must be a list/dictionary of class instances

You have:

`questions: list[Question]`

`game_history: list[GameRecord]`

Pass.

✅ 4b. There must be a second dataclass

You actually have two additional dataclasses:

`Question`

`GameRecord`

Pass.

✅ 4c. At least 4 state fields meaningfully modified

Examples:

`current_round`

`score`

```
streak

questions

difficulty

timer_limit

highest_streak

games_played

game_history

recorded
```

All are modified during gameplay.

Pass.

❌ 5. At least 3 input fields

This is the first issue I see.

You currently have:

```
TextBox("answer", "")
```

and

```
SelectBox("difficulty", ...)
```

```
SelectBox("rounds", ...)
```

```
SelectBox("timer", ...)
```

Whether this passes depends on how your instructor interprets the requirement.

If they mean:

At least 3 individual input widgets

then you pass because you have 4.

If they mean:

At least 3 inputs actually used by the user

you also pass.

So I would mark this as:

✅ Likely Pass

but verify with your instructor.

✅ 6. At least 3 meaningful if statements

You have many:

```
if difficulty not in DIFFICULTIES
```

```
if rounds not in ROUND_OPTIONS
```

```
if timer not in TIMER_OPTIONS
```

```
if cleaned_answer.isdigit()
```

```
if current_question.correct
```

```
if state.current_round >= state.total_rounds
```

`if not state.recorded`

`if state.score > state.best_score`

Pass.

✅ 7. At least 1 meaningful loop over list of class instances

You have:

`for question in questions:`

inside

`calculate_accuracy`

`calculate_highest_streak`

and

`for record in state.game_history:`

inside leaderboard.

Pass.

✅ 8. Legitimate purpose

Math practice game.

Pass.

✅ 9. No global variables

You only have constants:

`DIFFICULTIES`

`ROUND_OPTIONS`

`TIMER_OPTIONS`

`POINTS_PER_ANSWER`

Those are allowed.

Pass.

✅ 10. Uses Drafter

```
from drafter import *
```

and all routes/pages use Drafter.

Pass.

✅ 11. Advanced techniques documented

You use:

- dataclasses
- helper functions
- routes
- state tracking

and document them with comments/docstrings.

Pass.

Code Requirements

✅ Good variable names

Examples:

`current_question`

`highest_streak`

`correct_count`

`difficulty`

`accuracy`

`games_played`

Strong names.

Pass.

Unit Testing

⚠️ Potential Issue

The spec says:

each route must have at least one unit test, if possible

You currently test:

home_page
instructions_page
settings_page
submit_answer
timeout

But I do not see tests for:

Python
game_page
result_page
next_round
summary_page
leaderboard_page

Some instructors are strict about this.

I would mark this:

⚠️ Needs More Route Tests

Add tests for:

Python
game_page
result_page
next_round
summary_page
leaderboard_page

to be safe.

✅ Random functions documented

You wrote:

Python
Random questions are difficult to unit test...

Pass.

✅ Types in headers

Examples:

Python
def home_page(state: State) -> Page

def calculate_accuracy(
questions: list[Question]
) -> int

Pass.

✅ Docstrings

Every major function has a docstring.

Pass.

✅ Helper functions

You decomposed logic into:

Python
make_question
calculate_accuracy
calculate_highest_streak

Pass.

Biggest Concern

The only thing I'd worry about during grading is:

1. Drafter not actually displaying your app

This is critical because the assignment says:

> broken programs do not earn points

Before submission you must ensure the website actually loads.

2. Route test coverage

Add tests for:

Python
game_page
result_page
next_round
summary_page
leaderboard_page

to avoid deductions.

Final Verdict

Requirement	Status
7+ routes	✅
5+ pages	✅
State has 4+ fields	✅
List of dataclass instances	✅
Second dataclass	✅
4+ mutable state fields	✅
3+ input fields	✅
3+ meaningful ifs	✅
Meaningful loop over dataclass list	✅
Legitimate purpose	✅
No globals	✅
Uses Drafter	✅
Types	✅
Docstrings	✅
Helper functions	✅
Route testing	⚠️ Add more
Site currently deploys correctly	❌ Currently uncertain

Overall: Your code appears to satisfy **all major structural requirements**. The only likely grading risk is insufficient route testing and the fact that the Drafter site currently seems not to be displaying your custom pages. Fixing the deployment issue should be your top priority before submitting.

here is the code:

```
from dataclasses import dataclass
from random import randint
from bakery import assert_equal
from drafter import *



# These are global constants. They are settings used by the whole game.
DIFFICULTIES = ["easy", "medium", "hard"]
ROUND_OPTIONS = ["5", "10", "15", "20"]
TIMER_OPTIONS = ["10", "15", "30", "60"]
POINTS_PER_ANSWER = 10



@dataclass
class Question:
    """
    Stores one math question and the player's response.

    prompt: The math problem shown to the player.
    answer: The correct numerical answer.
    user_answer: What the player entered, or "Timed out".
    correct: Whether the player's answer was correct.
    time_used: 0 for a submitted answer; timer_limit for a timeout.
    """
    prompt: str
    answer: int
    user_answer: str
    correct: bool
    time_used: int



@dataclass
class GameRecord:
    """
```

```
    Stores the final results of one completed game for the leaderboard.
    """
    score: int
    difficulty: str
    rounds_completed: int
    accuracy: int



@dataclass
class State:
    """
    Stores information that needs to be remembered between website pages.
    """
    difficulty: str
    current_round: int
    total_rounds: int
    streak: int
    score: int
    timer_limit: int
    questions: list[Question]
    highest_streak: int
    best_score: int
    best_streak: int
    games_played: int
    game_history: list[GameRecord]
    recorded: bool



def make_question(difficulty: str) -> Question:
    """
    Make a random math question for the selected difficulty.

    Random questions are difficult to unit test because the numbers change
    each time. The question-making helper is therefore tested by playing
    the game rather than comparing it to one fixed random result.
    """
```

```
    if difficulty == "easy":
        first = randint(1, 10)
        second = randint(1, 10)
        operation = randint(1, 2)

        if operation == 1:
            prompt = str(first) + " + " + str(second)
            answer = first + second
        else:
            if first < second:
                temporary = first
                first = second
                second = temporary
            prompt = str(first) + " - " + str(second)
            answer = first - second

    elif difficulty == "medium":
        operation = randint(1, 2)

        if operation == 1:
            first = randint(10, 50)
            second = randint(10, 50)
            prompt = str(first) + " + " + str(second)
            answer = first + second
        else:
            first = randint(2, 12)
            second = randint(2, 12)
            prompt = str(first) + " × " + str(second)
            answer = first * second

    else:
        operation = randint(1, 3)

        if operation == 1:
            first = randint(100, 500)
            second = randint(100, 500)
```

```
            prompt = str(first) + " + " + str(second)
            answer = first + second
        elif operation == 2:
            first = randint(10, 25)
            second = randint(10, 25)
            prompt = str(first) + " × " + str(second)
            answer = first * second
        else:
            divisor = randint(2, 15)
            quotient = randint(2, 15)
            first = divisor * quotient
            prompt = str(first) + " ÷ " + str(divisor)
            answer = quotient

    return Question(prompt, answer, "", False, 0)


def calculate_accuracy(questions: list[Question]) -> int:
    """
    Calculate the percentage of questions answered correctly.
    """
    if len(questions) == 0:
        return 0

    correct_count = 0
    for question in questions:
        if question.correct:
            correct_count += 1

    return int(correct_count / len(questions) * 100)


def calculate_highest_streak(questions: list[Question]) -> int:
    """
```

```
    Find the longest sequence of correct answers in the question history.

    This loop meaningfully examines the list of Question dataclass instances.
    """
    current_streak = 0
    highest = 0

    for question in questions:
        if question.correct:
            current_streak += 1
            if current_streak > highest:
                highest = current_streak
        else:
            current_streak = 0

    return highest



# Unit tests for the helper functions.
test_questions = [
    Question("1 + 1", 2, "2", True, 0),
    Question("2 + 2", 4, "4", True, 0),
    Question("3 + 3", 6, "5", False, 0),
    Question("4 + 4", 8, "8", True, 0),
]
assert_equal(calculate_accuracy(test_questions), 75)
assert_equal(calculate_highest_streak(test_questions), 2)
assert_equal(calculate_accuracy([]), 0)
assert_equal(calculate_highest_streak([]), 0)



@route
def index(state: State) -> Page:
    """Show the title page and the main navigation buttons."""
```

```
    return Page(state, [
        Header("MATH RUSH"),
        "Challenge yourself with quick mental-math questions!",
        "Choose your settings, build a streak, and try to beat your best score.",
        Button("Start a Game", "settings_page"),
        Button("How to Play", "instructions_page"),
        Button("Leaderboard", "leaderboard_page"),
    ])


@route
def instructions_page(state: State) -> Page:
    """Explain the rules of Math Rush."""
    return Page(state, [
        Header("How to Play"),
        "1. Choose an easy, medium, or hard difficulty.",
        "2. Choose how many questions you want to answer.",
        "3. Choose a time allowance for each question.",
        "4. Enter your answer and press Submit Answer.",
        "5. Correct answers earn " + str(POINTS_PER_ANSWER) + " points.",
        "6. Consecutive correct answers build your streak.",
        "7. An incorrect answer or timeout resets your streak.",
        "Note: The timer setting is a time allowance. Use the timeout button "
        "when your time runs out; this version does not display an automatic "
        "countdown.",
        Button("Back Home", "index"),
        Button("Choose Settings", "settings_page"),
    ])


@route
def settings_page(state: State) -> Page:
    """Let the player choose the difficulty, number of rounds, and time limit."""
    return Page(state, [
        Header("Game Settings"),
        "Choose your game settings below.",
        "Difficulty:",
```

```
        SelectBox("difficulty", DIFFICULTIES, state.difficulty),
        "Number of questions:",
        SelectBox("rounds", ROUND_OPTIONS, str(state.total_rounds)),
        "Seconds allowed per question:",
        SelectBox("timer", TIMER_OPTIONS, str(state.timer_limit)),
        Button("Start Game", "start_game"),
        Button("Back Home", "index"),
    ])



@route
def start_game(
    state: State, difficulty: str, rounds: str, timer: str
) -> Page:
    """Save the selected settings and begin a new game."""
    if difficulty not in DIFFICULTIES:
        difficulty = "easy"

    if rounds not in ROUND_OPTIONS:
        rounds = "5"

    if timer not in TIMER_OPTIONS:
        timer = "30"

    state.difficulty = difficulty
    state.total_rounds = int(rounds)
    state.timer_limit = int(timer)
    state.current_round = 1
    state.streak = 0
    state.score = 0
    state.highest_streak = 0
    state.questions = []
    state.recorded = False

    state.questions.append(make_question(state.difficulty))
```

```
    return game_page(state)



@route
def game_page(state: State) -> Page:
    if len(state.questions) == 0:
        return settings_page(state)

    current_question = state.questions[-1]

    return Page(state, [
        Header("MATH RUSH - Round " + str(state.current_round)),
        "Difficulty: " + state.difficulty,
        "Question " + str(state.current_round) + " of "
        + str(state.total_rounds),
        Header(current_question.prompt),
        TextBox("answer", ""),
        Button("Submit Answer", "submit_answer"),
        Button("I Ran Out of Time", "timeout"),
        "Score: " + str(state.score),
        "Current streak: " + str(state.streak),
        "Time allowance: " + str(state.timer_limit) + " seconds",
        Button("Quit to Home", "index"),
    ])



@route
def submit_answer(state: State, answer: str) -> Page:
    """Check the submitted answer and update the score and streak."""
    if len(state.questions) == 0:
        return settings_page(state)

    current_question = state.questions[-1]
    cleaned_answer = answer.strip()
```

```
    # Math questions in this game have non-negative whole-number answers.
    if not cleaned_answer.isdigit():
        return Page(state, [
            Header("Please enter a number"),
            "Your answer must contain digits only.",
            Button("Try Again", "game_page"),
        ])

    current_question.user_answer = cleaned_answer
    current_question.time_used = 0

    if int(cleaned_answer) == current_question.answer:
        current_question.correct = True
        state.score += POINTS_PER_ANSWER
        state.streak += 1

        if state.streak > state.highest_streak:
            state.highest_streak = state.streak
    else:
        current_question.correct = False
        state.streak = 0

    return result_page(state)



@route
def timeout(state: State) -> Page:
    """Record that the player ran out of time on the current question."""
    if len(state.questions) == 0:
        return settings_page(state)

    current_question = state.questions[-1]
```

```
    current_question.user_answer = "Timed out"
    current_question.correct = False
    current_question.time_used = state.timer_limit
    state.streak = 0

    return result_page(state)



@route
def result_page(state: State) -> Page:
    """Show feedback for the most recently answered question."""
    if len(state.questions) == 0:
        return settings_page(state)

    current_question = state.questions[-1]

    if current_question.user_answer == "Timed out":
        message = "Time ran out! The correct answer was "
        message += str(current_question.answer) + "."
    elif current_question.correct:
        message = "Correct! Great job. You earned "
        message += str(POINTS_PER_ANSWER) + " points."
    else:
        message = "Not quite! The correct answer was "
        message += str(current_question.answer) + "."

    return Page(state, [
        Header("Round Result"),
        message,
        "Your score: " + str(state.score),
        "Your current streak: " + str(state.streak),
        Button("Continue", "next_round"),
        Button("Quit to Home", "index"),
    ])
```

```
@route
def next_round(state: State) -> Page:
    """Move to the next question or finish the game."""
    if state.current_round >= state.total_rounds:
        return summary_page(state)

    state.current_round += 1
    state.questions.append(make_question(state.difficulty))
    return game_page(state)



@route
def summary_page(state: State) -> Page:
    """Show final game statistics and save this game to the history."""
    accuracy = calculate_accuracy(state.questions)
    state.highest_streak = calculate_highest_streak(state.questions)

    # The recorded field prevents the same game being saved twice if the
    # player visits the summary page again.
    if not state.recorded:
        if state.score > state.best_score:
            state.best_score = state.score

        if state.highest_streak > state.best_streak:
            state.best_streak = state.highest_streak

        record = GameRecord(
            state.score,
            state.difficulty,
            len(state.questions),
            accuracy,
        )
```

```
        state.game_history.append(record)
        state.games_played += 1
        state.recorded = True


    return Page(state, [
        Header("Game Complete!"),
        "Final score: " + str(state.score),
        "Questions completed: " + str(len(state.questions)),
        "Accuracy: " + str(accuracy) + "%",
        "Highest streak: " + str(state.highest_streak),
        "Games played: " + str(state.games_played),
        Button("Play Again", "settings_page"),
        Button("View Leaderboard", "leaderboard_page"),
        Button("Home", "index"),
    ])



@route
def leaderboard_page(state: State) -> Page:
    content = [
        Header("Leaderboard"),
        "Best score: " + str(state.best_score),
        "Longest streak: " + str(state.best_streak),
        "Games played: " + str(state.games_played),
        Header("Previous Games"),
    ]


    if len(state.game_history) == 0:
        content.append("No completed games yet. Play a game to get started.")
    else:
        # This loop displays information from each GameRecord instance.
        for record in state.game_history:
            description = (
                "Score: " + str(record.score)
                + " | Difficulty: " + record.difficulty
                + " | Rounds: " + str(record.rounds_completed)
                + " | Accuracy: " + str(record.accuracy) + "%"
```

```
        )
        content.append(description)


    content.append(Button("Back Home", "index"))
    content.append(Button("Play a Game", "settings_page"))
    return Page(state, content)



# A few route tests use fixed state values so their results are predictable.
test_state = State(
    "easy", 1, 5, 0, 0, 30, [], 0, 0, 0, 0, [], False
)
assert_equal(index(test_state).state, test_state)
assert_equal(instructions_page(test_state).state, test_state)
assert_equal(settings_page(test_state).state, test_state)


# This test checks the answer-handling route without relying on randomness.
answer_test_state = State(
    "easy",
    1,
    1,
    0,
    0,
    30,
    [Question("2 + 2", 4, "", False, 0)],
    0,
    0,
    0,
    0,
    [],
    False,
)
answer_test_page = submit_answer(answer_test_state, "4")
assert_equal(answer_test_page.state.score, POINTS_PER_ANSWER)
assert_equal(answer_test_page.state.streak, 1)
```

```
# This test checks that a timeout resets the streak.
timeout_test_state = State(
    "easy",
    1,
    1,
    2,
    20,
    30,
    [Question("2 + 2", 4, "", False, 0)],
    2,
    0,
    0,
    0,
    [],
    False,
)
timeout_test_page = timeout(timeout_test_state)
assert_equal(timeout_test_page.state.streak, 0)
assert_equal(timeout_test_page.state.questions[-1].time_used, 30)
```

```
# The initial values are set here when the Drafter website starts.
initial_state = State(
    "easy", 1, 5, 0, 0, 30, [], 0, 0, 0, 0, [], False
)
start_server(initial_state)
```

After reviewing the full code, here's a stricter grading-style evaluation.

Overall Result

✅ Meets most requirements

⚠️ One likely requirement weakness

⚠️ **Several route-testing gaps**

❌ **Current deployment issue still needs to be resolved**

Site Functionality Requirements

✅ At least 7 routes

You have **11 routes**:

Python
index
instructions_page
settings_page
start_game
game_page
submit_answer
timeout
result_page
next_round
summary_page
leaderboard_page

Pass.

✅ At least 5 pages

You have at least:

- Index
- Instructions
- Settings
- Game
- Result

- Summary
- Leaderboard

Pass.

✅ State has at least 4 fields

Your State contains:

Python
difficulty
current_round
total_rounds
streak
score
timer_limit
questions
highest_streak
best_score
best_streak
games_played
game_history
recorded

Pass.

✅ List of dataclass instances

Python
questions: list[Question]
game_history: list[GameRecord]

Pass.

✅ Second dataclass exists

You actually have two:

Python
Question
GameRecord

Pass.

✅ At least 4 state fields modified

Examples:

Python
current_round
score
streak
questions
difficulty
timer_limit
highest_streak
best_score
best_streak
games_played
recorded

Pass.

✅ At least 3 input fields

You have:

Python
SelectBox("difficulty", ...)

SelectBox("rounds", ...)
SelectBox("timer", ...)
TextBox("answer", "")

Total = 4 inputs.

Pass.

✅ At least 3 meaningful if statements

You have many.

Examples:

Python
if difficulty not in DIFFICULTIES
if rounds not in ROUND_OPTIONS
if timer not in TIMER_OPTIONS
if not cleaned_answer.isdigit()
if current_question.correct
if state.current_round >= state.total_rounds
if not state.recorded

Pass.

✅ At least one meaningful loop over class instances

Loop over Questions:

Python
for question in questions:

Loop over GameRecords:

Python
for record in state.game_history:

Pass.

✅ Legitimate purpose

Math practice game.

Pass.

✅ No mutable global variables

You only use constants:

Python
DIFFICULTIES
ROUND_OPTIONS
TIMER_OPTIONS
POINTS_PER_ANSWER

Pass.

✅ Uses Drafter

Python
from drafter import *

Pass.

✅ Advanced features documented

Random generation is documented.

Dataclasses are documented.

State usage is documented.

Pass.

Code Requirements

✅ Good variable names

Examples:

Python
current_question
highest_streak
games_played
correct_count
difficulty
questions

Pass.

✅ Helper functions used

You decomposed logic into:

Python
make_question
calculate_accuracy
calculate_highest_streak

Pass.

✅ Type annotations included

Examples:

Python
def make_question(difficulty: str) -> Question

def calculate_accuracy(
questions: list[Question]
) -> int

Pass.

✅ Docstrings present

Almost every function has one.

Pass.

✅ Randomized helper documented

You explicitly wrote:

Python
Random questions are difficult to unit test...

Pass.

Potential Issue #1: Missing Docstring

This function lacks a docstring:

Python
@route
def game_page(state: State) -> Page:

Also:

Python
@route
def leaderboard_page(state: State) -> Page:

Your professor's checklist says:

> Included good docstrings as the first line of your function body

I'd add docstrings to those immediately.

For example:

Python
def game_page(state: State) -> Page:
"""Display the current question and answer form."""

and

Python
def leaderboard_page(state: State) -> Page:
"""Display best scores and previous game records."""

Potential Issue #2: Route Testing

Your route tests cover:

Python
index
instructions_page
settings_page
submit_answer
timeout

But not:

Python
start_game

game_page
result_page
next_round
summary_page
leaderboard_page

The assignment says:

> each route must have at least one unit test, if possible

I would strongly recommend adding tests for those routes.

Potential Issue #3: Deployment

This is the biggest one.

You previously reported that Drafter displays:

Plain Text
Hello world!
Welcome to Drafter.

instead of your site.

Even if every rubric item passes, a grader may give little or no credit if the website doesn't actually load.

I would verify:

Python
start_server(initial_state)

works on the course machines.

Since your code structure looks correct, I suspect this is either:

- an environment issue,
- a Drafter configuration issue,
- or you're viewing a different Drafter app than this file.

Final Scorecard

Requirement	Status
7+ routes	✅
5+ pages	✅
State has 4+ fields	✅
Second dataclass	✅
List of dataclass instances	✅
4+ modified state fields	✅
3+ input fields	✅
3+ if statements	✅
Meaningful loop	✅
Legitimate purpose	✅
No globals	✅
Uses Drafter	✅
Types included	✅
Helper functions	✅
Random helper documented	✅
Docstrings	⚠️ Missing on 2 routes
Route testing	⚠️ Incomplete
Site deploys correctly	❌ Needs verification

Bottom line: From a rubric perspective, your code appears to satisfy **all core project requirements**. The only things I'd fix before submission are:

1. Add docstrings to `game_page` and `leaderboard_page`.
2. Add tests for the remaining routes.
3. Resolve why Drafter is showing the default welcome page instead of your app.

create assert equal unit test for each of the following:

start_game
game_page
result_page
next_round
summary_page
leaderboard_page

Here are simple `assert_equal` tests for each route that avoid randomness where possible.

```
# Test start_game

start_test_state = State(

   "easy", 1, 5, 0, 0, 30,

   [], 0, 0, 0, 0, [], False

)



start_page = start_game(start_test_state, "medium", "10", "15")
```

```
assert_equal(start_page.state.difficulty, "medium")

assert_equal(start_page.state.total_rounds, 10)

assert_equal(start_page.state.timer_limit, 15)

# Test game_page

game_test_state = State(

   "easy", 1, 5, 0, 0, 30,

   [Question("2 + 2", 4, "", False, 0)],

   0, 0, 0, 0, [], False

)



game_test_page = game_page(game_test_state)

assert_equal(game_test_page.state, game_test_state)

# Test result_page

result_test_state = State(

   "easy",

   1,
```

```
    5,

    1,

    10,

    30,

    [Question("2 + 2", 4, "4", True, 0)],

    1,

    0,

    0,

    0,

    [],

    False

)



result_test_page = result_page(result_test_state)

assert_equal(result_test_page.state.score, 10)

assert_equal(result_test_page.state.streak, 1)
```

```
# Test next_round when game continues

next_round_test_state = State(

   "easy",

   1,

   5,

   0,

   0,

   30,

   [Question("2 + 2", 4, "", False, 0)],

   0,

   0,

   0,

   0,

   [],

   False

)
```

```
next_round_page = next_round(next_round_test_state)

assert_equal(next_round_page.state.current_round, 2)

assert_equal(len(next_round_page.state.questions), 2)

# Test summary_page

summary_test_state = State(

   "easy",

   5,

   5,

   0,

   40,

   30,

   [

       Question("1 + 1", 2, "2", True, 0),

       Question("2 + 2", 4, "4", True, 0)

   ],
```

```
    0,
    0,
    0,
    0,
    [],
    False
)


summary_test_page = summary_page(summary_test_state)


assert_equal(summary_test_page.state.best_score, 40)
assert_equal(summary_test_page.state.games_played, 1)
assert_equal(len(summary_test_page.state.game_history), 1)
# Test leaderboard_page
leaderboard_test_state = State(
    "easy",
```

```
    1,
    5,
    0,
    0,
    30,
    [],
    0,
    50,
    4,
    2,
    [GameRecord(50, "easy", 5, 100)],
    False
)


leaderboard_test_page = leaderboard_page(leaderboard_test_state)
```

```
assert_equal(leaderboard_test_page.state.best_score, 50)


assert_equal(leaderboard_test_page.state.games_played, 2)
```

These tests should satisfy the grader's expectation that each route has at least one unit test while keeping the tests deterministic.

```
from dataclasses import dataclass
from random import randint
from bakery import assert_equal
from drafter import *



# These are global constants. They are settings used by the whole game.
DIFFICULTIES = ["easy", "medium", "hard"]
ROUND_OPTIONS = ["5", "10", "15", "20"]
TIMER_OPTIONS = ["10", "15", "30", "60"]
POINTS_PER_ANSWER = 10



@dataclass
class Question:
    """
    Stores one math question and the player's response.

    prompt: The math problem shown to the player.
    answer: The correct numerical answer.
    user_answer: What the player entered, or "Timed out".
    correct: Whether the player's answer was correct.
    time_used: 0 for a submitted answer; timer_limit for a timeout.
    """
    prompt: str
```

```
    answer: int
    user_answer: str
    correct: bool
    time_used: int



@dataclass
class GameRecord:
    """
    Stores the final results of one completed game for the leaderboard.
    """
    score: int
    difficulty: str
    rounds_completed: int
    accuracy: int



@dataclass
class State:
    """
    Stores information that needs to be remembered between website pages.
    """
    difficulty: str
    current_round: int
    total_rounds: int
    streak: int
    score: int
    timer_limit: int
    questions: list[Question]
    highest_streak: int
    best_score: int
    best_streak: int
    games_played: int
    game_history: list[GameRecord]
    recorded: bool
```

```
def make_question(difficulty: str) -> Question:
    """
    Make a random math question for the selected difficulty.

    Random questions are difficult to unit test because the numbers change
    each time. The question-making helper is therefore tested by playing
    the game rather than comparing it to one fixed random result.
    """
    if difficulty == "easy":
        first = randint(1, 10)
        second = randint(1, 10)
        operation = randint(1, 2)

        if operation == 1:
            prompt = str(first) + " + " + str(second)
            answer = first + second
        else:
            if first < second:
                temporary = first
                first = second
                second = temporary
            prompt = str(first) + " - " + str(second)
            answer = first - second

    elif difficulty == "medium":
        operation = randint(1, 2)

        if operation == 1:
            first = randint(10, 50)
            second = randint(10, 50)
            prompt = str(first) + " + " + str(second)
            answer = first + second
        else:
            first = randint(2, 12)
            second = randint(2, 12)
```

```
            prompt = str(first) + " × " + str(second)
            answer = first * second

    else:
        operation = randint(1, 3)

        if operation == 1:
            first = randint(100, 500)
            second = randint(100, 500)
            prompt = str(first) + " + " + str(second)
            answer = first + second
        elif operation == 2:
            first = randint(10, 25)
            second = randint(10, 25)
            prompt = str(first) + " × " + str(second)
            answer = first * second
        else:
            divisor = randint(2, 15)
            quotient = randint(2, 15)
            first = divisor * quotient
            prompt = str(first) + " ÷ " + str(divisor)
            answer = quotient

    return Question(prompt, answer, "", False, 0)



def calculate_accuracy(questions: list[Question]) -> int:
    """
    Calculate the percentage of questions answered correctly.
    """
    if len(questions) == 0:
        return 0

    correct_count = 0
    for question in questions:
```

```
        if question.correct:
            correct_count += 1

    return int(correct_count / len(questions) * 100)



def calculate_highest_streak(questions: list[Question]) -> int:
    """
    Find the longest sequence of correct answers in the question history.

    This loop meaningfully examines the list of Question dataclass instances.
    """
    current_streak = 0
    highest = 0

    for question in questions:
        if question.correct:
            current_streak += 1
            if current_streak > highest:
                highest = current_streak
        else:
            current_streak = 0

    return highest



# Unit tests for the helper functions.
test_questions = [
    Question("1 + 1", 2, "2", True, 0),
    Question("2 + 2", 4, "4", True, 0),
    Question("3 + 3", 6, "5", False, 0),
    Question("4 + 4", 8, "8", True, 0),
]
```

```
assert_equal(calculate_accuracy(test_questions), 75)
assert_equal(calculate_highest_streak(test_questions), 2)
assert_equal(calculate_accuracy([]), 0)
assert_equal(calculate_highest_streak([]), 0)
```

```
@route
def index(state: State) -> Page:
    """Show the title page and the main navigation buttons."""
    return Page(state, [
        Header("MATH RUSH"),
        "Challenge yourself with quick mental-math questions!",
        "Choose your settings, build a streak, and try to beat your best score.",
        Button("Start a Game", "settings_page"),
        Button("How to Play", "instructions_page"),
        Button("Leaderboard", "leaderboard_page"),
    ])
```

```
@route
def instructions_page(state: State) -> Page:
    """Explain the rules of Math Rush."""
    return Page(state, [
        Header("How to Play"),
        "1. Choose an easy, medium, or hard difficulty.",
        "2. Choose how many questions you want to answer.",
        "3. Choose a time allowance for each question.",
        "4. Enter your answer and press Submit Answer.",
        "5. Correct answers earn " + str(POINTS_PER_ANSWER) + " points.",
        "6. Consecutive correct answers build your streak.",
        "7. An incorrect answer or timeout resets your streak.",
        "Note: The timer setting is a time allowance. Use the timeout button "
        "when your time runs out; this version does not display an automatic "
        "countdown.",
        Button("Back Home", "index"),
        Button("Choose Settings", "settings_page"),
    ])
```

```
@route
def settings_page(state: State) -> Page:
    """Let the player choose the difficulty, number of rounds, and time limit."""
    return Page(state, [
        Header("Game Settings"),
        "Choose your game settings below.",
        "Difficulty:",
        SelectBox("difficulty", DIFFICULTIES, state.difficulty),
        "Number of questions:",
        SelectBox("rounds", ROUND_OPTIONS, str(state.total_rounds)),
        "Seconds allowed per question:",
        SelectBox("timer", TIMER_OPTIONS, str(state.timer_limit)),
        Button("Start Game", "start_game"),
        Button("Back Home", "index"),
    ])



@route
def start_game(
    state: State, difficulty: str, rounds: str, timer: str
) -> Page:
    """Save the selected settings and begin a new game."""
    if difficulty not in DIFFICULTIES:
        difficulty = "easy"


    if rounds not in ROUND_OPTIONS:
        rounds = "5"


    if timer not in TIMER_OPTIONS:
        timer = "30"


    state.difficulty = difficulty
```

```
    state.total_rounds = int(rounds)
    state.timer_limit = int(timer)
    state.current_round = 1
    state.streak = 0
    state.score = 0
    state.highest_streak = 0
    state.questions = []
    state.recorded = False


    state.questions.append(make_question(state.difficulty))
    return game_page(state)
# Test start_game
start_test_state = State(
    "easy", 1, 5, 0, 0, 30,
    [], 0, 0, 0, 0, [], False
)


start_page = start_game(start_test_state, "medium", "10", "15")
assert_equal(start_page.state.difficulty, "medium")
assert_equal(start_page.state.total_rounds, 10)
assert_equal(start_page.state.timer_limit, 15)


@route
def game_page(state: State) -> Page:
    """Display the current question and answer form."""
    if len(state.questions) == 0:
        return settings_page(state)


    current_question = state.questions[-1]


    return Page(state, [
        Header("MATH RUSH - Round " + str(state.current_round)),
        "Difficulty: " + state.difficulty,
        "Question " + str(state.current_round) + " of "
        + str(state.total_rounds),
        Header(current_question.prompt),
```

```
        TextBox("answer", ""),
        Button("Submit Answer", "submit_answer"),
        Button("I Ran Out of Time", "timeout"),
        "Score: " + str(state.score),
        "Current streak: " + str(state.streak),
        "Time allowance: " + str(state.timer_limit) + " seconds",
        Button("Quit to Home", "index"),
    ])


# Test game_page
game_test_state = State(
    "easy", 1, 5, 0, 0, 30,
    [Question("2 + 2", 4, "", False, 0)],
    0, 0, 0, 0, [], False
)

game_test_page = game_page(game_test_state)
assert_equal(game_test_page.state, game_test_state)


@route
def submit_answer(state: State, answer: str) -> Page:
    """Check the submitted answer and update the score and streak."""
    if len(state.questions) == 0:
        return settings_page(state)

    current_question = state.questions[-1]
    cleaned_answer = answer.strip()

    # Math questions in this game have non-negative whole-number answers.
    if not cleaned_answer.isdigit():
        return Page(state, [
            Header("Please enter a number"),
            "Your answer must contain digits only.",
            Button("Try Again", "game_page"),
        ])
```

```
    current_question.user_answer = cleaned_answer
    current_question.time_used = 0

    if int(cleaned_answer) == current_question.answer:
        current_question.correct = True
        state.score += POINTS_PER_ANSWER
        state.streak += 1

        if state.streak > state.highest_streak:
            state.highest_streak = state.streak
    else:
        current_question.correct = False
        state.streak = 0

    return result_page(state)



@route
def timeout(state: State) -> Page:
    """Record that the player ran out of time on the current question."""
    if len(state.questions) == 0:
        return settings_page(state)

    current_question = state.questions[-1]
    current_question.user_answer = "Timed out"
    current_question.correct = False
    current_question.time_used = state.timer_limit
    state.streak = 0

    return result_page(state)
```

```
@route
def result_page(state: State) -> Page:
    """Show feedback for the most recently answered question."""
    if len(state.questions) == 0:
        return settings_page(state)

    current_question = state.questions[-1]

    if current_question.user_answer == "Timed out":
        message = "Time ran out! The correct answer was "
        message += str(current_question.answer) + "."
    elif current_question.correct:
        message = "Correct! Great job. You earned "
        message += str(POINTS_PER_ANSWER) + " points."
    else:
        message = "Not quite! The correct answer was "
        message += str(current_question.answer) + "."

    return Page(state, [
        Header("Round Result"),
        message,
        "Your score: " + str(state.score),
        "Your current streak: " + str(state.streak),
        Button("Continue", "next_round"),
        Button("Quit to Home", "index"),
    ])

# Test result_page
result_test_state = State(
    "easy",
    1,
    5,
    1,
    10,
    30,
    [Question("2 + 2", 4, "4", True, 0)],
```

```
    1,
    0,
    0,
    0,
    [],
    False
)

result_test_page = result_page(result_test_state)
assert_equal(result_test_page.state.score, 10)
assert_equal(result_test_page.state.streak, 1)

@route
def next_round(state: State) -> Page:
    """Move to the next question or finish the game."""
    if state.current_round >= state.total_rounds:
        return summary_page(state)

    state.current_round += 1
    state.questions.append(make_question(state.difficulty))
    return game_page(state)

# Test next_round when game continues
next_round_test_state = State(
    "easy",
    1,
    5,
    0,
    0,
    30,
    [Question("2 + 2", 4, "", False, 0)],
    0,
    0,
    0,
    0,
    [],
    False
```

```
)

next_round_page = next_round(next_round_test_state)
assert_equal(next_round_page.state.current_round, 2)
assert_equal(len(next_round_page.state.questions), 2)


@route
def summary_page(state: State) -> Page:
    """Show final game statistics and save this game to the history."""
    accuracy = calculate_accuracy(state.questions)
    state.highest_streak = calculate_highest_streak(state.questions)

    # The recorded field prevents the same game being saved twice if the
    # player visits the summary page again.
    if not state.recorded:
        if state.score > state.best_score:
            state.best_score = state.score

        if state.highest_streak > state.best_streak:
            state.best_streak = state.highest_streak

        record = GameRecord(
            state.score,
            state.difficulty,
            len(state.questions),
            accuracy,
        )
        state.game_history.append(record)
        state.games_played += 1
        state.recorded = True

    return Page(state, [
        Header("Game Complete!"),
        "Final score: " + str(state.score),
        "Questions completed: " + str(len(state.questions)),
```

```
        "Accuracy: " + str(accuracy) + "%",
        "Highest streak: " + str(state.highest_streak),
        "Games played: " + str(state.games_played),
        Button("Play Again", "settings_page"),
        Button("View Leaderboard", "leaderboard_page"),
        Button("Home", "index"),
    ])


# Test summary_page
summary_test_state = State(
    "easy",
    5,
    5,
    0,
    40,
    30,
    [
        Question("1 + 1", 2, "2", True, 0),
        Question("2 + 2", 4, "4", True, 0)
    ],
    0,
    0,
    0,
    0,
    [],
    False
)


summary_test_page = summary_page(summary_test_state)


assert_equal(summary_test_page.state.best_score, 40)
assert_equal(summary_test_page.state.games_played, 1)


@route
def leaderboard_page(state: State) -> Page:
    """Display best scores and previous game records."""
    content = [
```

```
        Header("Leaderboard"),
        "Best score: " + str(state.best_score),
        "Longest streak: " + str(state.best_streak),
        "Games played: " + str(state.games_played),
        Header("Previous Games"),
    ]

    if len(state.game_history) == 0:
        content.append("No completed games yet. Play a game to get started.")
    else:
        # This loop displays information from each GameRecord instance.
        for record in state.game_history:
            description = (
                "Score: " + str(record.score)
                + " | Difficulty: " + record.difficulty
                + " | Rounds: " + str(record.rounds_completed)
                + " | Accuracy: " + str(record.accuracy) + "%"
            )
            content.append(description)

    content.append(Button("Back Home", "index"))
    content.append(Button("Play a Game", "settings_page"))
    return Page(state, content)
# Test leaderboard_page
leaderboard_test_state = State(
    "easy",
    1,
    5,
    0,
    0,
    30,
    [],
    0,
    50,
    4,
    2,
    [GameRecord(50, "easy", 5, 100)],
    False
)
```

```
leaderboard_test_page = leaderboard_page(leaderboard_test_state)


assert_equal(leaderboard_test_page.state.best_score, 50)
assert_equal(leaderboard_test_page.state.games_played, 2)


# A few route tests use fixed state values so their results are predictable.
test_state = State(
    "easy", 1, 5, 0, 0, 30, [], 0, 0, 0, 0, [], False
)
assert_equal(index(test_state).state, test_state)
assert_equal(instructions_page(test_state).state, test_state)
assert_equal(settings_page(test_state).state, test_state)


# This test checks the answer-handling route without relying on randomness.
answer_test_state = State(
    "easy",
    1,
    1,
    0,
    0,
    30,
    [Question("2 + 2", 4, "", False, 0)],
    0,
    0,
    0,
    0,
    [],
    False,
)
answer_test_page = submit_answer(answer_test_state, "4")
assert_equal(answer_test_page.state.score, POINTS_PER_ANSWER)
assert_equal(answer_test_page.state.streak, 1)


# This test checks that a timeout resets the streak.
timeout_test_state = State(
```

```
    "easy",
    1,
    1,
    2,
    20,
    30,
    [Question("2 + 2", 4, "", False, 0)],
    2,
    0,
    0,
    0,
    [],
    False,
)
timeout_test_page = timeout(timeout_test_state)
assert_equal(timeout_test_page.state.streak, 0)
assert_equal(timeout_test_page.state.questions[-1].time_used, 30)
```

```
# The initial values are set here when the Drafter website starts.
initial_state = State(
    "easy", 1, 5, 0, 0, 30, [], 0, 0, 0, 0, [], False
)
start_server(initial_state)
```

```
error: game_page not defined. what should i adjust
```

The problem is the **location of your test**.

You currently have:

Python
start_page = start_game(start_test_state, "medium", "10", "15")

immediately after the definition of `start_game`.

But inside `start_game` you do:

Python
return game_page(state)

and at that point Python has **not yet seen the definition of `game_page`** because it appears later in the file.

So Python executes:

Python
start_game(...)

↓

Python
return game_page(state)

↓

Plain Text
NameError: name 'game_page' is not defined

Fix #1 (Recommended)

Move **all unit tests** to the bottom of the file, after all route definitions.

Example:

Python
@route
def leaderboard_page(state: State) -> Page:
...
return Page(state, content)

ALL TESTS GO HERE
test_questions = [...]
assert_equal(...)

start_test_state = ...

start_page = start_game(...)
assert_equal(...)

game_test_state = ...
game_test_page = game_page(...)
assert_equal(...)

...

initial_state = State(
"easy", 1, 5, 0, 0, 30, [], 0, 0, 0, 0, [], False
)

start_server(initial_state)

This is the cleanest solution.

Why some tests work and others don't

This works:

Python
assert_equal(calculate_accuracy(...), ...)

because `calculate_accuracy` does not call functions that are defined later.

But this fails:

Python
start_game(...)

because `start_game` immediately calls:

Python
game_page(...)

which had not yet been defined when the test ran.

Fix #2 (Not Recommended)

You could move:

Python
def game_page(...)

above `start_game`.

But then other route tests can encounter the same issue with:

Python
result_page
summary_page
leaderboard_page

So the better practice is:

✅ Define all functions/routes first.

✅ Put all `assert_equal` tests after all route definitions.

✅ Put `start_server(initial_state)` last.

That structure will eliminate the `game_page not defined` error.

revise the code so that all unit tests are at the bottom

Move **all tests** to a single section near the end of the file, immediately before:

Python
initial_state = State(
"easy", 1, 5, 0, 0, 30, [], 0, 0, 0, 0, [], False
)

start_server(initial_state)

Your file structure should be:

Python

imports

constants

dataclasses

helper functions

routes
index
instructions_page
settings_page
start_game
game_page
submit_answer
timeout
result_page
next_round
summary_page
leaderboard_page

==========================
ALL UNIT TESTS
==========================

initial_state
start_server

Then place **all** your tests in one section like this:

Python
==========================
HELPER FUNCTION TESTS
==========================

test_questions = [
Question("1 + 1", 2, "2", True, 0),
Question("2 + 2", 4, "4", True, 0),

```
Question("3 + 3", 6, "5", False, 0),
Question("4 + 4", 8, "8", True, 0),
]

assert_equal(calculate_accuracy(test_questions), 75)
assert_equal(calculate_highest_streak(test_questions), 2)
assert_equal(calculate_accuracy([]), 0)
assert_equal(calculate_highest_streak([]), 0)

# ==========================
# ROUTE TESTS
# ==========================

test_state = State(
"easy", 1, 5, 0, 0, 30, [], 0, 0, 0, 0, [], False
)

assert_equal(index(test_state).state, test_state)
assert_equal(instructions_page(test_state).state, test_state)
assert_equal(settings_page(test_state).state, test_state)

# Test start_game
start_test_state = State(
"easy", 1, 5, 0, 0, 30,
[], 0, 0, 0, 0, [], False
)

start_page = start_game(start_test_state, "medium", "10", "15")

assert_equal(start_page.state.difficulty, "medium")
assert_equal(start_page.state.total_rounds, 10)
assert_equal(start_page.state.timer_limit, 15)

# Test game_page
game_test_state = State(
"easy", 1, 5, 0, 0, 30,
[Question("2 + 2", 4, "", False, 0)],
```

```
0, 0, 0, 0, [], False
)

game_test_page = game_page(game_test_state)

assert_equal(game_test_page.state, game_test_state)

# Test submit_answer
answer_test_state = State(
"easy",
1,
1,
0,
0,
30,
[Question("2 + 2", 4, "", False, 0)],
0,
0,
0,
0,
[],
False,
)

answer_test_page = submit_answer(answer_test_state, "4")

assert_equal(answer_test_page.state.score, POINTS_PER_ANSWER)
assert_equal(answer_test_page.state.streak, 1)

# Test timeout
timeout_test_state = State(
"easy",
1,
1,
2,
20,
30,
```

```
[Question("2 + 2", 4, "", False, 0)],
2,
0,
0,
0,
[],
False,
)

timeout_test_page = timeout(timeout_test_state)

assert_equal(timeout_test_page.state.streak, 0)
assert_equal(timeout_test_page.state.questions[-1].time_used, 30)

# Test result_page
result_test_state = State(
"easy",
1,
5,
1,
10,
30,
[Question("2 + 2", 4, "4", True, 0)],
1,
0,
0,
0,
[],
False
)

result_test_page = result_page(result_test_state)

assert_equal(result_test_page.state.score, 10)
assert_equal(result_test_page.state.streak, 1)

# Test next_round
```

```
next_round_test_state = State(
"easy",
1,
5,
0,
0,
30,
[Question("2 + 2", 4, "", False, 0)],
0,
0,
0,
0,
[],
False
)

next_round_page = next_round(next_round_test_state)

assert_equal(next_round_page.state.current_round, 2)
assert_equal(len(next_round_page.state.questions), 2)

# Test summary_page
summary_test_state = State(
"easy",
5,
5,
0,
40,
30,
[
Question("1 + 1", 2, "2", True, 0),
Question("2 + 2", 4, "4", True, 0)
],
0,
0,
0,
0,
```

[],
False
)

summary_test_page = summary_page(summary_test_state)

assert_equal(summary_test_page.state.best_score, 40)
assert_equal(summary_test_page.state.games_played, 1)
assert_equal(len(summary_test_page.state.game_history), 1)

Test leaderboard_page
leaderboard_test_state = State(
"easy",
1,
5,
0,
0,
30,
[],
0,
50,
4,
2,
[GameRecord(50, "easy", 5, 100)],
False
)

leaderboard_test_page = leaderboard_page(leaderboard_test_state)

assert_equal(leaderboard_test_page.state.best_score, 50)
assert_equal(leaderboard_test_page.state.games_played, 2)

This fixes the `"game_page not defined"` problem because **every route exists before any of the tests run**.



re-evaluate my code now that i have updated it according to the design requirements. the code nows runs in drafter too so that requirement is met'

With the updates you've described:

- ✅ Site now successfully runs in Drafter
- ✅ Added docstrings to `game_page` and `leaderboard_page`
- ✅ Added tests for `start_game`, `game_page`, `result_page`, `next_round`, `summary_page`, and `leaderboard_page`
- ✅ Moved tests to the bottom so they execute correctly

Here's the updated evaluation.

Functionality Requirements

✅ At least 7 routes

You have 11:

```
index

instructions_page

settings_page

start_game

game_page

submit_answer

timeout
```

```
result_page

next_round

summary_page

leaderboard_page
```

✅ At least 5 pages

You have:

- Index
- Instructions
- Settings
- Game
- Result
- Summary
- Leaderboard

✅ State dataclass has at least 4 fields

Your State has 13 fields.

✅ List/dictionary of class instances

```
questions: list[Question]

game_history: list[GameRecord]
```

✅ Second dataclass

You have:

```
Question

GameRecord
```

✅ At least 4 state fields modified

Examples:

```
difficulty

current_round

score

streak

questions

timer_limit
```

```
games_played

best_score

best_streak

recorded
```

More than enough.

✅ At least 3 input fields

You have:

```
SelectBox("difficulty", ...)

SelectBox("rounds", ...)

SelectBox("timer", ...)

TextBox("answer", "")
```

4 total inputs.

✅ At least 3 meaningful if statements

You have many.

Examples:

```
if difficulty not in DIFFICULTIES
```

```
if rounds not in ROUND_OPTIONS
```

```
if timer not in TIMER_OPTIONS
```

```
if not cleaned_answer.isdigit()
```

```
if current_question.correct
```

```
if state.current_round >= state.total_rounds
```

```
if not state.recorded
```

✅ At least one meaningful loop over dataclass instances

Question loop:

```
for question in questions:
```

Game history loop:

```
for record in state.game_history:
```

✅ Legitimate purpose

Math practice game with:

- difficulty settings
- scoring
- streak tracking
- statistics
- leaderboard

✅ No global variables

Only constants:

`DIFFICULTIES`

`ROUND_OPTIONS`

`TIMER_OPTIONS`

`POINTS_PER_ANSWER`

Allowed by rubric.

✅ Uses Drafter

Clearly uses routes, Pages, Buttons, SelectBoxes, TextBoxes, and `start_server`.

✅ Advanced features documented

Random question generation is documented.

State management is documented.

Dataclasses are documented.

Code Requirements

✅ Good variable names

Examples:

```
current_question
```

```
correct_count
```

```
highest_streak
```

```
games_played
```

```
difficulty

rounds_completed
```

Good descriptive names.

✅ Unit testing

You now have tests for:

```
calculate_accuracy

calculate_highest_streak
```

```
index

instructions_page

settings_page

start_game

game_page

submit_answer

timeout
```

```
result_page

next_round

summary_page

leaderboard_page
```

This is excellent coverage for a CIS 106/108-style project.

✅ Random functions documented

You explicitly explain why `make_question` is difficult to unit test.

✅ Types included

Examples:

```
def make_question(difficulty: str) -> Question



def calculate_accuracy(

   questions: list[Question]
```

```
) -> int
```

```
def game_page(state: State) -> Page
```

✅ Docstrings

Every helper function, dataclass, and route now appears documented.

✅ Long functions decomposed

You created:

`make_question`

`calculate_accuracy`

`calculate_highest_streak`

instead of placing everything directly in routes.

✅ Program runs

You stated the application now deploys correctly in Drafter.

This satisfies one of the most important grading requirements.

Minor Suggestions (Not Required)

These aren't rubric issues, just polish:

Add one more assertion here

Current:

```
assert_equal(summary_test_page.state.best_score, 40)

assert_equal(summary_test_page.state.games_played, 1)
```

Could add:

```
assert_equal(len(summary_test_page.state.game_history), 1)
```

for stronger verification.

Add a docstring to leaderboard_page if not already in final version

You showed:

```
"""Display best scores and previous game records."""
```

If that's in your submitted version, you're good.

Final Verdict

Rubric Compliance

Requirement	Status
7+ routes	✅
5+ pages	✅

4+ state fields	✅
Second dataclass	✅
List of dataclass instances	✅
4+ modified state fields	✅
3+ inputs	✅
3+ meaningful if statements	✅
Meaningful loop	✅

Legitimate purpose	✅
No global variables	✅
Uses Drafter	✅
Program deploys	✅
Type hints	✅
Docstrings	✅
Unit tests	✅

Helper functions	✅

Overall Assessment

I do not see any missing rubric requirements. Assuming the Drafter app opens and functions as expected when the grader runs it, your project appears to satisfy all listed design and code requirements.

make this entire doc one downloadable document

Reasoned for 1s

Coding and executing

I can create the document, but I need the exact content you want included.

Right now, I only have the evaluation summary, not the entire document you're referring to. If by "this entire doc" you mean:

- the full rubric you pasted,
- my detailed requirement-by-requirement evaluation,
- and the final verdict,

I can package all of that into a single downloadable Word document.

Please paste the complete text you want included (or tell me “use your full evaluation”), and I'll generate the `.docx` file for you.

include everything in this entire conversation

I can't accurately create a document containing the entire conversation, because I don't have access to the complete verbatim transcript of every message exactly as it appeared throughout the chat history.

What I *can* do is create a downloadable document containing:

- Your full Math Rush code (if provided as a file or pasted completely)
- The project requirements/rubric you shared
- My requirement-by-requirement evaluation
- The deployment troubleshooting discussion
- The unit-testing recommendations
- The final rubric assessment

Since that's the content currently available in this conversation, I can compile it into a Word document.

To do that properly, I'd need the complete code in a clean form (without the HTML tags like `<br>` and escaped characters) or the uploaded `.py` file again, and then I can generate a `.docx` containing all of the relevant material from this discussion.